

An Efficient Layer Normalization Training Module With Dynamic Quantization for Transformers

Haikuo Shao^{1b}, Aotao Wang, and Zhongfeng Wang^{1b}, *Fellow, IEEE*

Abstract—Layer normalization (LN) function is widely adopted in Transformer-based neural networks. The efficient training of Transformers on personal devices is attracting attention for data privacy and latency concerns. However, the critical LN function involves extreme outliers for quantization, as well as hardware-unfriendly square-root and division operations, posing resource challenges for training deployment on the edge. This brief proposes an efficient LN training architecture with algorithm and hardware co-optimization. Specifically, we present a dynamic quantized algorithm based on integer arithmetics to smooth outliers for sufficient training accuracy. Then, we develop a reconfigurable hardware architecture to efficiently support various operations during LN training, with a vector-wise pipelined dataflow to improve hardware efficiency further. Experimental results show that our architecture achieves up to 0.25 and 1.0 Giga input per Second (GinS) in throughput at FPGA and ASIC platforms, respectively, outperforming prior works.

Index Terms—Transformer, training, softmax, activation function, architecture, algorithm.

I. INTRODUCTION

OVER the past few years, Transformer-based [1] neural networks have achieved remarkable success in various domains of deep learning [2]. With the expanding application scope of Transformers, it has become both prevalent and crucial to retrain the models on edge devices using personal data, considering the data transmission latency and privacy. However, the substantial and complicated computations of Transformers pose challenges to their deployment, especially training on resource-constrained edge devices [3].

Transformer-based models primarily consist of linear operations (fully connected layer and matrix multiplication) and non-linear functions (such as Softmax and layer normalization [1]). Recent works for Transformers training acceleration focus mainly on enhancing linear operations [4], [5] and Softmax [6], [7], [8] efficiency. However, as the above operations are optimized, the computational cost proportion of Layer Normalization (LN) grows inevitably. Through performance profiling experiments on 3090 Graphics Processing Unit (GPU) platform, the LN runtime proportion increases on average

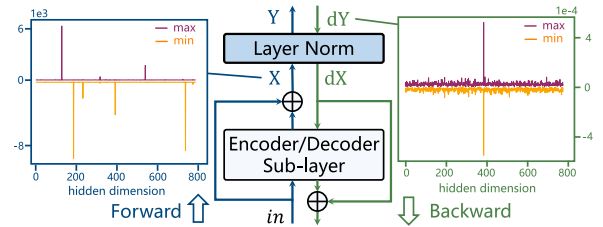


Fig. 1. LN in Transformer and its computational dataflow during training. There are obvious outliers in the last hidden dimension of X and dY , which are LN's input tensor of forward and backward propagation, respectively.

from 6% to 15% as linear operations are optimized from single floating-point to 8-bit integers for the Transformer-based BERT [9] model deployment, becoming a considerable overhead.

LN in Transformer is applied after each sub-layer (e.g., self-attention and feed-forward networks) to stabilize training dynamics by normalizing activations. The main challenges of LN edge deployment stem from the extreme outliers along the hidden dimension in input activations (X) and output gradients (dY), as illustrated in Fig. 1. Besides, LN involves complex square-root and division operations. These issues make it highly sensitive to quantization and challenging for efficient hardware implementation.

Previous works have primarily focused on optimizing LN function in Transformer inference. Some algorithms [10], [11], [12] proposed quantized LN based on 32-bit integers. Several hardware solutions [13], [14] have also been presented to enhance the hardware efficiency with integer widths more than 19 bits. However, long-width integers still do not provide sufficient efficiency for hardware deployment. Other works [15], [16] developed hardware approximation methods with look-up tables (LUT) for square-root and division operations in LN, without thorough optimization. Besides, the above works focused on inference-only scenarios. Compared with inference, training tasks involve different computation patterns and broader data distributions. Prior research [17], [18], [19], [20] on training optimization has focused on batch normalization (BN) in convolutional neural networks (CNNs). However, LN has different computational paradigms and formulations compared to BN, making previous designs inapplicable.

In this brief, we present an efficient training architecture for LN functions in Transformers. The main contributions are:

- 1) We thoroughly analyze the differences between LN and BN and propose an LN training algorithm with dynamic 12-bit quantization and square-root approximation, which mitigates the impact of outliers and maintains training accuracy across various Transformer models.
- 2) Based on the algorithm, an efficient architecture with vector-wise pipelined dataflow is developed to support LN

Received 4 May 2025; revised 23 June 2025; accepted 16 July 2025. Date of publication 22 July 2025; date of current version 29 August 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4400600, and in part by the Postgraduate Research & Practice Innovation Program of Jiangsu Province under Grant KYCX24_0149. This brief was recommended by Associate Editor G. He. (Corresponding author: Zhongfeng Wang.)

Haikuo Shao and Aotao Wang are with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: hkshao@smail.nju.edu.cn).

Zhongfeng Wang is with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China, and also with the School of Integrated Circuits, Sun Yat-sen University, Shenzhen 518107, China (e-mail: zfwang@nju.edu.cn).

Digital Object Identifier 10.1109/TCSII.2025.3591633

1549-7747 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Lenovo Group LTD. Downloaded on August 06, 2026 at 05:49:18 UTC from IEEE Xplore. Restrictions apply.

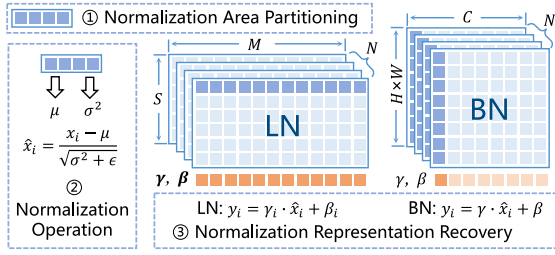


Fig. 2. Comparisons of LN ($\mathbf{X} \in \mathbb{R}^{N \times S \times M}$, S is sequence length and M is hidden dimension) and BN ($\mathbf{X} \in \mathbb{R}^{N \times C \times H \times W}$, C , H , and W are channel number, height, and width of feature map). N is the training batch size.

computations in both forward and backward phases during training, with superior performance to prior works.

II. BACKGROUND AND RELATED WORKS

A. Layer Normalization and Batch Normalization

In Transformer-based neural networks, LN is typically applied to the output tensor $\mathbf{X} \in \mathbb{R}^{N \times S \times M}$ of each sub-layer in both the encoder and decoder, including the self-attention and feed-forward layers. According to [21], the normalization framework can be divided into three components: normalization area partitioning (NAP), normalization operation (NOP), and normalization representation recovery (NRR), as depicted in Fig. 2. The NAP in LN is a vector $\mathbf{x} = (x_1, x_2, \dots, x_M)$ in $\mathbf{X}$ that corresponds to the embedding of a token in the input sequence. During NOP, the mean μ and variance σ^2 of $\mathbf{x}$ are first computed through:

$$\mu = \frac{1}{M} \sum_{i=1}^M x_i, \quad \sigma^2 = \frac{1}{M} \sum_{i=1}^M (x_i - \mu)^2. \quad (1)$$

Assuming $\lambda = 1/\sqrt{\sigma^2 + \epsilon}$, the normalized result $\hat{\mathbf{x}}$ is computed as Eq. (2), where ϵ is a small constant.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} = \lambda \cdot (x_i - \mu). \quad (2)$$

Then, the output vector $\mathbf{y}$ of LN is obtained by NRR in (3):

$$y_i = \gamma_i \hat{x}_i + \beta_i. \quad (3)$$

The weight $\boldsymbol{\gamma} \in \mathbb{R}^M$ and bias $\boldsymbol{\beta} \in \mathbb{R}^M$ are two learnable parameters. Note that $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ in LN are applied to all NAP areas in $\mathbf{X}$.

As illustrated in Fig. 2, the differences between LN and BN exist in the NAP and NRR stages. The NAP of BN is $N \times H \times W$ activation values in a mini-batch. The normalization is performed independently along the channel dimension. During NRR, γ and β are scalars for each area, meaning that all elements in the current area share the same γ and β .

The training process is divided into forward propagation (FP) and backward propagation (BP). In the BP phase, the gradient of training loss w.r.t $\mathbf{y}$, $\frac{\partial \ell}{\partial \mathbf{y}}$ (denoted as $\mathbf{dy}$ here), is propagated from subsequent layers of LN. Firstly, since $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ in LN are applied to all normalization areas in $\mathbf{X}$, their gradients can be computed through the chain rule:

$$d\gamma_i = \sum_{j=1}^{S \cdot N} dy_{ji} \times \hat{x}_{ji}, \quad d\beta_i = \sum_{j=1}^{S \cdot N} dy_{ji}. \quad (4)$$

According to [22], the gradient $d\mathbf{x}$ can be derived as follows:

$$d\hat{x}_i = \gamma_i dy_i, \quad d\sigma^2 = \frac{-1}{2} \lambda^3 \cdot \sum_{i=1}^M ((x_i - \mu) d\hat{x}_i),$$

TABLE I
FINE-TUNING RESULTS OF BERT ON GLUE BENCHMARK

Method	MNLI	QQP	QNLI	SST2	MRPC	STSBB
Baseline- <i>fp32</i>	84.86	91.01	91.32	93.00	84.31	87.77
8-Block	46.87	84.40	50.54	82.11	68.38	19.25
16-Block	74.85	84.58	79.48	84.17	68.73	25.70
Frozen- $\gamma\beta$	84.80	91.15	91.18	92.66	83.82	87.96

$$\begin{aligned} d\mu &= -\lambda \sum d\hat{x}_i + \frac{-2}{M} \sum ((x_i - \mu) \cdot d\sigma^2), \\ dx_i &= \lambda d\hat{x}_i + \frac{2(x_i - \mu)}{M} \cdot d\sigma^2 + \frac{1}{M} d\mu, \\ dx_i &= \lambda \gamma_i dy_i - \frac{\lambda \hat{x}_i}{M} \sum (\gamma_i \hat{x}_i dy_i) - \frac{\lambda}{M} \sum (\gamma_i dy_i) \\ &\quad + \frac{\lambda}{M^2} \sum (\hat{x}_i \sum (\gamma_i \hat{x}_i dy_i)). \end{aligned} \quad (5)$$

Since M is typically larger than 512 in Transformers, experiments observe that the magnitude of the fourth term in Eq. (5) is more than three orders smaller than that of the other terms. Therefore, dx_i can be simplified as follows:

$$dx_i = \lambda \left(\gamma_i dy_i - \hat{x}_i \cdot \frac{1}{M} \sum (\gamma_i \hat{x}_i dy_i) - \frac{1}{M} \sum (\gamma_i dy_i) \right). \quad (6)$$

B. Related Normalization Training Works

State-of-the-art normalization works mainly focused on the optimizations of BN in CNNs' training. Bactran [17] designed architectures to support BN training based on the range BN (RBN). ACBN [18], OPN [19], and PBN [20] proposed approximately calculated BNs for on-device training. The key idea of these works is to approximate the mean and/or variance using the maximum and minimum values in the mini-batch. For example, ACBN uses Eq. (7) to compute the mean and standard deviation, where x_i^n is the element in the n th feature map and c is an adjustment term.

$$\tilde{\mu} = \frac{\sum \max(x_i^n) + \sum \min(x_i^n)}{2N}, \quad \tilde{\sigma} = c \frac{\sum \max(x_i^n) - \sum \min(x_i^n)}{2N}. \quad (7)$$

However, such ideas are inspired by the intrinsic Gaussian distributions [18] in each feature map of N-Batch CNNs' training. Unfortunately, LN in Transformers does not have this characteristic due to different NAP patterns. We conducted preliminary experiments to divide the NAP of LN into several blocks and fine-tuned the BERT-base model on various tasks using this method. As listed in Table I, the model accuracy significantly decreased. In addition to data distribution, CNNs in these works are usually trained from scratch, which can better adapt to the approximated BN. Transformers prefer to fine-tune the pre-trained models.

III. DYNAMIC QUANTIZED LN TRAINING ALGORITHM

Symmetric uniform quantization is a widely adopted method due to its simplicity and computational efficiency. In this approach, the floating-point tensor $\mathbf{x}$ is linearly mapped to a quantized integer representation $\mathbf{x}_q$ using function $Q(\cdot)$:

$$\mathbf{x}_q = Q(\mathbf{x}, s, b) = \text{clip}\left(\text{round}\left(\frac{\mathbf{x}}{s}\right), -2^{b-1}, 2^{b-1} - 1\right), \quad (8)$$

where b is the quantization bit-width, and s is the scaling factor determined by the range of $\mathbf{x}$.

As shown in Fig. 1, the input tensors for both the FP and BP of LN exhibit significant variation along the hidden dimension. If $\mathbf{x}$ is quantized using a fixed scaling factor, the presence of extreme outliers will lead to intolerable quantization loss. A possible solution is applying different factors to different positions in the hidden dimension. However, this method will

Algorithm 1 Dynamic Quantized LN Training Algorithm**FP-Input:** $\mathbf{x} = (x_1, x_2, \dots, x_M)$, $\boldsymbol{\gamma} \in \mathbb{R}^M$, $\boldsymbol{\beta} \in \mathbb{R}^M$ **FP-Output:** $\mathbf{y} = (y_1, y_2, \dots, y_M)$

1: $\mathbf{x}_q \leftarrow \text{DPTQ}(\mathbf{x})$, $\mathbf{s} = (s_1, s_2, \dots, s_M)$, $s_i \in \{s^{(1)}, s^{(2)}\}$
 2: $\mu_q \leftarrow \text{mean}(\mathbf{x}_q \cdot \mathbf{s}/s^{(1)})$, $s_\mu = s^{(1)}$
 3: $v_q \leftarrow \text{mean}(\mathbf{x}_q^2 \cdot (\mathbf{s}/s^{(1)})^2)$, $s_v = (s^{(1)})^2$
 4: $\sigma_q^2 \leftarrow (v_q - \mu_q^2)$, $\lambda_q \leftarrow \text{QISR}(\sigma_q^2, \epsilon_q)$
 5: $\hat{\mathbf{x}}_q \leftarrow (\mathbf{x}_q \cdot \mathbf{s}/s^{(1)} - \mu_q) \times \lambda_q$
 6: $\mathbf{y}_q \leftarrow \hat{\mathbf{x}}_q \times \boldsymbol{\gamma}_q + \boldsymbol{\beta}_q$, $\boldsymbol{\gamma}_q = \text{Q}(\boldsymbol{\gamma})$, $\boldsymbol{\beta}_q = \text{Q}(\boldsymbol{\beta})$

BP-Input: $\mathbf{dy} = (dy_1, dy_2, \dots, dy_M)$, $\boldsymbol{\gamma}_q$, $\hat{\mathbf{x}}_q$, $\boldsymbol{\gamma}_q \hat{\mathbf{x}}_q$ saved in **FP****BP-Output:** $\mathbf{dx} = (dx_1, dx_2, \dots, dx_M)$

7: $\mathbf{dy}_q \leftarrow \text{DPTQ}(\mathbf{dy})$, $\mathbf{s}' = (s'_1, s'_2, \dots, s'_M)$, $s'_i \in \{s'^{(1)}, s'^{(2)}\}$
 8: $\mu'_q \leftarrow \text{mean}(\boldsymbol{\gamma}_q \mathbf{dy}_q)$, $\boldsymbol{\gamma}_q \mathbf{dy}_q = \mathbf{dy}_q \cdot \mathbf{s}'/s'^{(1)} \times \boldsymbol{\gamma}_q$
 9: $\mu''_q \leftarrow \text{mean}(\mathbf{dy}_q \cdot \mathbf{s}'/s'^{(1)} \times \boldsymbol{\gamma}_q \hat{\mathbf{x}}_q)$
 10: $\mathbf{dx}_q \leftarrow \lambda_q \times (\boldsymbol{\gamma}_q \mathbf{dy}_q - \hat{\mathbf{x}}_q \times \mu''_q - \mu'_q)$

induce the calculation of μ and σ^2 still in the floating-point domain, resulting in a high hardware overhead.

We propose a coarse-grained yet efficient method, dynamic power-of-two quantization (**DPTQ**), to tackle this problem. Specifically, we apply different factors, smaller $s^{(1)}$ and larger $s^{(2)}$, to non-outliers and outliers in $\mathbf{x}$, respectively. $s^{(1)}$ preserves the accuracy of non-outliers while $s^{(2)}$ retains the amplitude information of outliers during accumulation. The outliers are determined by a pre-defined threshold τ . Given the bit-width b , $\tau = (2^{b-1} - 1) \cdot s^{(1)}$, where $s^{(1)}$ is selected to cover the dynamic range of typical $\mathbf{x}$ and $\mathbf{dy}$ non-outliers within a calibration batch. **DPTQ** is defined as follows:

$$\text{DPTQ}(x_i) = \begin{cases} \text{Q}(x_i, s^{(2)}, b) & \text{if } |x_i| > \tau, \\ \text{Q}(x_i, s^{(1)}, b) & \text{if } |x_i| \leq \tau. \end{cases} \quad (9)$$

We also employ power-of-two quantization to constrain $s = 2^k$, where k is an integer. **DPTQ** can divide the computation of μ and σ^2 into two groups and combine their results based on the difference between $s^{(1)}$ and $s^{(2)}$ with a simple shift operation. Gradient $\mathbf{dy}$ was also quantized with **DPTQ**, while other variables without obvious outliers apply normal power-of-two quantization.

The fully quantized LN (**QLN**) training process is summarized in Algorithm 1. Overall, the computation of $\mathbf{y}$ and $\mathbf{dx}$ can be divided into two stages: (1) statistics computation (lines 1~4, 7~9) and (2) affine transformations (lines 5, 6, 10).

Based on the variance decomposition formula $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, σ^2 is reformulated as $\text{mean}(x^2) - \mu^2$ to decouple data dependency between μ and σ^2 . Next, λ_q is obtained from σ_q^2 with quantized inverse square root (**QISR**) method. Specifically, input $u = \sigma_q^2 + \epsilon_q$ is scaled by $\tilde{u} = u/2^{2r}$ ($\gg 2r$ -bit in hardware) into range $[0.25, 1)$, where r is an integer. Since $\frac{1}{\sqrt{\tilde{u}}} \in [2, 1)$ when $\tilde{u} \in [0.25, 1)$, we use 12-piece-wise linear approximation method to fit $\frac{1}{\sqrt{\tilde{u}}}$. Then, the approximated $\frac{1}{\sqrt{\tilde{u}}}$ is re-scaled by $\frac{1}{\sqrt{\tilde{u}}} \cdot 2^r$ to get $\frac{1}{\sqrt{u}}$.

Unlike BN, where $d\gamma$ and $d\beta$ are intermediate results of $\mathbf{dx}$ computation [18], $d\gamma$ and $d\beta$ in LN require the accumulation of $\mathbf{dy}$ across all NAPs in $\mathbf{X}$ as derived in Eq. (4), which is independent of $\mathbf{dx}$ and brings extra computational overhead. Some studies [23] observed that most model information is learned by MHA and FFN blocks during training. We freeze $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ in LN during full-precision BERT fine-tuning and find that their update is subtle and freezing them has a negligible influence on the model accuracy, as summarized in Table I.

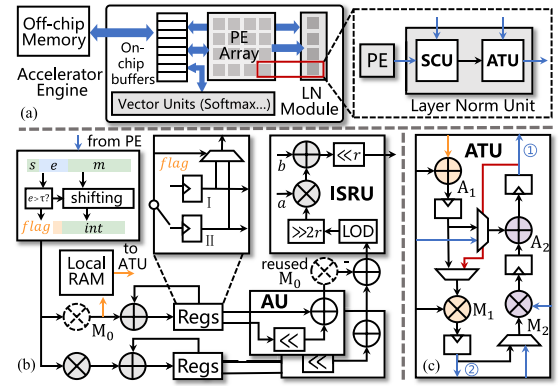


Fig. 3. (a) Overall architecture of Layer Norm Unit that contains (b) Statistics Computation Unit (SCU) and (c) Affine Transformation Unit (ATU).

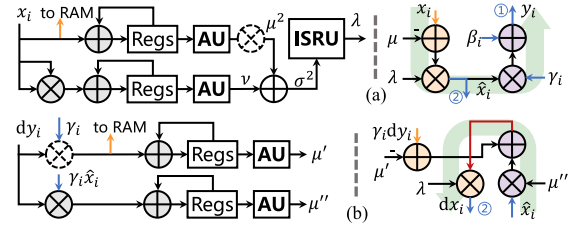


Fig. 4. The detailed dataflow arrangements for SCU and ATU in (a) FP and (b) BP phases during training.

IV. HARDWARE AND DATAFLOW DESIGN

Based on the dynamic quantized LN algorithm, the hardware architecture for LN training is presented in Fig. 3. The training accelerator engine primarily includes a Processing Element (PE) Array for matrix multiplication, an LN module, on-chip buffers for data exchange with off-chip memory, and vector units used for functions like Softmax and activation. The LN module is composed of parallel Layer Norm Units (**LNU**), and each LNU contains a Statistics Computation Unit (**SCU**) and an Affine Transformation Unit (**ATU**). As illustrated in Fig. 3(b), during the training process, the SCU receives results ($\mathbf{x}$ or $\mathbf{dy}$) from PE and converts the floating-point data to short integers according to the exponent e and the threshold τ . The 1-bit *flag* is used to indicate whether x_i or dy_i are outliers and determines which register is chosen for accumulation in two registers (Regs). The accumulated results of the two Regs are then sent to the Align Unit (**AU**), where they are reduced through simple shift and add operations based on the difference between scaling factors. The Inverse Square Root Unit (**ISRU**) performs the proposed QISR algorithm with a leading-one detector (**LOD**) to determine the factor for scaling before and rescaling after linear approximation ($ax + b$). After SCU, the data and statistics are transmitted to ATU to finish element-wise affine transformations as Fig. 3(c).

Dataflow arrangement: The specific dataflow during FP and BP phases of LN are shown in Fig. 4. Note that the multiplier M_0 is used to calculate μ^2 and $\gamma_i dy_i$ in FP and BP, respectively. Data x_i and $\gamma_i dy_i$ are temporarily saved in the local RAM for the next stage's computations in ATU, and ISRU is used in FP to compute λ . Internal multiplexers configure ATU to support different operations in FP and BP phases following the right part of Fig. 4. To further improve the training throughput and hardware utilization, the operations in SCU and ATU are organized in a pipelined manner to fit the data dependencies in these two stages, as shown in Fig. 5. Taking FP as an example,

TABLE II
BERT, GPT-2, AND ViT FINE-TUNING PERFORMANCES AND COMPARISONS ON LANGUAGE AND IMAGE PROCESSING BENCHMARKS

Model	BERT-110M Accuracy $\uparrow$								GPT-2-117M Acc. & PPL $\downarrow$				ViT-110M Accuracy		
Task	MNLI	QQP	QNLI	SST2	MRPC	STS B	Avg.		PTB	WikiText	CIFAR-10	CIFAR-100	ImageNet		
Baseline- <i>fp32</i>	84.86	91.01	91.32	93.00	84.31	87.77	88.71		45.65	17.72	42.59	21.13	99.81	91.60	83.77
QLN	84.81	91.08	91.29	92.32	83.09	87.78	88.40		44.69	19.70	41.71	23.53	99.03	91.30	83.65
QLN frozen- $\gamma\beta$	84.65	90.40	91.32	92.66	83.57	88.14	88.46		44.67	19.67	41.74	23.11	99.01	91.73	83.61
Ref- <i>int16</i>	35.33	63.18	49.46	79.01	68.38	22.03	52.90		33.37	54.66	29.46	69.44	30.98	42.84	25.58

* The training batch size is 32 for BERT, 8 for GPT-2, and 256 for ViT. We use the same learning rate (2×10^{-5} for BERT and ViT, 5×10^{-5} for GPT-2) and Adam optimizer for different schemes and fine-tune BERT/GPT-2 for 3 epochs, ViT for 5 epochs.

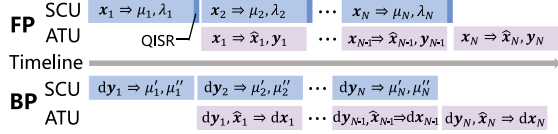


Fig. 5. The vector-wise pipelined schedule for FP and BP phases.

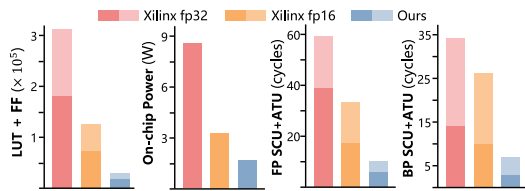


Fig. 6. LUT, FF, power, and latency comparisons with baselines.

vector x_1 is first sent to SCU to process all its elements to obtain statistical values μ_1 and λ_1 . Then x_1 performs affine transformations in ATU, meanwhile, vector x_2 is processed in SCU. Since SCU and ATU stages have nearly the same vector latency, the vector-wise pipelined schedule can benefit overall $2\times$ throughput.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

Algorithm: We fine-tune encoder-based BERT on GLUE [9] (General Language Understanding Evaluation) benchmark with various downstream tasks. We also evaluate decoder-based GPT-2 on popular language modeling tasks and report the perplexities (PPL), including Penn Treebank (PTB) and WikiText-2 datasets [24]. Additionally, we apply the quantized algorithm to Vision Transformer (ViT) for image classification tasks on CIFAR and ImageNet [25] datasets. We compare the results of floating-point (*fp32*) and integer (*int16*) baselines with our quantized schemes (QLN) with and without frozen- $\gamma\beta$. The quantization bit-widths for multiplier and adder operands in Fig. 3 are 12-bit and 32-bit, respectively.

Hardware: The hardware architecture is coded with SystemVerilog and evaluated with both FPGA and ASIC platforms. For FPGA, the architecture contains 32 parallel LNUs and is synthesized by Vivado Design Suite and implemented on Xilinx ZCU102 FPGA at 250-MHz frequency. The ASIC implementation evaluated a single LNU using Synopsys Design Compiler and PrimeTime-PX power toolkit under TSMC 28-nm technology at 1-GHz. For different accelerator systems, the LNU parallelism can be flexibly configured to match the PE array specifications. Benefiting from the data fusion design between PE array and LN Module, increasing LNU parallelism does not pose extra off-chip bandwidth overhead.

B. Algorithm Accuracy for Fine-Tuning Transformers

The overall training results of different schemes are summarized in Table II. Compared with the *fp32* baseline, our

quantized QLN methods with frozen- $\gamma\beta$ can achieve equivalent performance on various downstream tasks, with less than 1% accuracy drop for BERT/ViT models and slight perplexity increase (also less than 1% loss in accuracy) for GPT-2. In contrast, Ref-*int16* in Table II employs 16-bit quantization for multiply operands without DPTQ. Under the same experimental configurations, it leads to significant accuracy loss and perplexity increase. The results demonstrate that our DPTQ and QISR schemes can achieve lower bit-widths than vanilla quantization while maintaining training accuracy.

C. Hardware Performance Evaluation

1) *Comparisons With FP Baselines:* The FPGA resource consumption of our quantized LN architecture is reported in Table III. We compare it with 32-bit and 16-bit floating point LN module using Xilinx Integrated IP libraries (Xilinx *fp32* and *fp16*) in Fig. 6. All three schemes employ 160 DSPs. Compared to Xilinx *fp32*, our design saves 90% Look-Up Table (LUT), 91% Flip Flop (FF) resources, and 80% power, while reducing 83% FP latency and 79% BP latency. In comparison with Xilinx *fp16*, our design also consumes 76% less LUT and 78% less FF. In an accelerator engine featuring a 32×32 *int8* PE array, replacing the *fp32* LN module with our design can contribute to 13.0% FP, 7.7% BP, and 7.4% overall training latency reduction on the BERT-base model.

2) *Versus Norm-Training Works:* As introduced in Section II-B, existing works [17], [18], [20] focused mainly on BN training. RBN [17] approximated the standard deviation based on the range of input values without quantization considerations. Compared with RBN, our design offers $2.5\times$ throughput and $8.5\times$ energy efficiency as summarized in Table III. More recent ACBN [18] and PBN [20] calculated both mean and standard deviation based on block-wise min and max values with 16-bit fixed-point operands. In comparison with them, our method achieves $1.3\times$ faster throughput and $2.5\times$ energy efficiency. Our benefits primarily stem from the lower quantization bit-width and the reconfigurable architecture design that efficiently supports both FP and BP computations.

3) *Versus LN-Inference Works:* NN-LUT [15] builds a trainable neural network and converts it into LUT operations to approximate nonlinear functions in Transformers, including exponential, activation, division, and square-root. The latter two functions can be used in LN inference. GQA-LUT [16] proposed a quantization-aware genetic algorithm to approximate nonlinear functions by LUT more accurately. However, they consider a variety of functions and do not dedicated to the overall optimization of LN and its BP implementation. Compared with them (Table IV), our method achieves $2.0\times\sim 5.4\times$ higher throughput with $4.4\times\sim 23.8\times$ better area-delay product (ADP) efficiency. Reference [14] presented a hardware-oriented LN inference approximation scheme based on 19-bit inputs. Our

TABLE III
COMPARISONS WITH OTHER NORMALIZATION-TRAINING WORKS

Design	RBN [17]	ACBN [18]	PBN [20]	Our work
FPGA Device	UV440	ZCU102	ZCU102	ZCU102
Frequency (MHz)	100	200	200	250
LUT	529671	43578	43675	17996
FF	255670	20367	44515	11768
DSP	1024	192	192	160
BRAM	264	80	96	96
Power (W)	5.612	3.291	-	1.659
Latency (ns)	10	5	5	4
Energy Efficiency (GinS/W)	0.57	1.94	-	4.82

TABLE IV
COMPARISONS WITH LN INFERENCE-ONLY WORKS

Design	NN-LUT [15]	GQA-LUT [16]	[14]	Our LNU
Training Support	X	X	X	✓
Area (μm^2)	16143	8040	3227	3684
Clock period (ns)	2.72	2.0	1.8	1.0
Latency (cycles)	2	1	1	1
Power (mW)	9.46	3.14	-	2.08
ADP ($\text{mm}^2 \times \text{ns}$)	0.878	0.161	0.058	0.037

* Area, power, and clock period are all scaled to 28nm technology.

† LUT designs are 16 entries of INT32 for sufficient accuracy.

method achieves $1.8\times$ higher throughput and $1.6\times$ ADP efficiency than [14] through lower bit-width design. Moreover, our multiplexer-based training module incurs only 12.7% area overhead compared with the vanilla inference-only LN design without BP support. The ISRU, used only in FP phase, occupies 16.0% of the area in LN module.

VI. CONCLUSION

In this brief, we proposed an efficient training module for layer normalization in Transformers with algorithm-hardware co-optimization. Firstly, we developed a hardware-friendly dynamic quantized LN training algorithm, maintaining sufficient training accuracy. Then, we designed a train-oriented hardware architecture and vector-wise pipelined dataflow, boosting training performance and minimizing resource consumption. Our work can be efficiently used as a function module in various Transformers' training and inference scenarios.

REFERENCES

- [1] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 1–15.
- [2] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, Nov. 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:235368340>
- [3] B. Zhuang, J. Liu, Z. Pan, H. He, Y. Weng, and C. Shen, "A survey on efficient training of transformers," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 6823–6831.
- [4] S.-H. Noh, J. J. Koo, S. Lee, J. Park, and J. Kung, "FlexBlock: A flexible DNN training accelerator with multi-mode block floating point support," *IEEE Trans. Comput.*, vol. 72, no. 9, pp. 2522–2535, Sep. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:247447624>
- [5] H. Shao, J. Lu, M. Wang, and Z. Wang, "An efficient training accelerator for transformers with hardware-algorithm co-optimization," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 11, pp. 1788–1801, Nov. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261929089>
- [6] T. Xia and S. Q. Zhang, "Hyft: A reconfigurable softmax accelerator with hybrid numeric format for both training and inference," in *Proc. Int. Symp. Low Power Electron. Design*, 2023, pp. 1–6. [Online]. Available: <https://api.semanticscholar.org/CorpusID:265352054>
- [7] F. Spagnolo, S. Perri, and P. Corsonello, "Aggressive approximation of the softmax function for power-efficient hardware implementations," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1652–1656, Mar. 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:244596354>
- [8] Z. Mei, H. Dong, Y. Wang, and H. Pan, "TEA-S: A tiny and efficient architecture for PLAC-based softmax in transformers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3594–3598, Sep. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258076056>
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguist.*, 2019, pp. 1–16.
- [10] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 5506–5518.
- [11] Z. Li and Q. Gu, "I-ViT: Integer-only quantization for efficient vision transformer inference," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2022, pp. 17019–17029. [Online]. Available: <https://api.semanticscholar.org/CorpusID:250264564>
- [12] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "FQ-ViT: Post-training quantization for fully quantized vision transformer," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 1–10. [Online]. Available: <https://api.semanticscholar.org/CorpusID:248300269>
- [13] Z. Zhao, R. Cao, K.-F. Un, W.-H. Yu, P.-I. Mak, and R. P. Martins, "An FPGA-based transformer accelerator using output block stationary dataflow for object recognition applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 1, pp. 281–285, Jan. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:251328521>
- [14] W. Li, D. Lyu, G. Wang, A. Hu, N. Xu, and G. He, "Hardware-oriented algorithms for softmax and layer normalization of large language models," *Sci. China Inf. Sci.*, vol. 67, Sep. 2024, Art. no. 200404. [Online]. Available: <https://api.semanticscholar.org/CorpusID:272859591>
- [15] J. Yu et al., "NN-LUT: Neural approximation of non-linear operations for efficient transformer inference," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, 2021, pp. 577–582. [Online]. Available: <https://api.semanticscholar.org/CorpusID:244909068>
- [16] P. Dong et al., "Genetic quantization-aware approximation for non-linear operations in transformers," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, 2024, pp. 1–6.
- [17] Y. Zhijie, W. Lei, L. Li, L. Shiming, G. Shasha, and W. Shuquan, "Bactran: A hardware batch normalization implementation for CNN training engine," *IEEE Embedded Syst. Lett.*, vol. 13, no. 1, pp. 29–32, Mar. 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:214253104>
- [18] B. Li, H. Wang, F. Luo, X. Zhang, H. Sun, and N. Zheng, "ACBN: Approximate calculated batch Normalization for efficient DNN on-device training processor," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, pp. 738–748, Jun. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258022616>
- [19] H. Dai, H. Wang, X. Zhang, and H. Sun, "Memory-efficient batch normalization by one-pass computation for on-device training," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 6, pp. 3186–3190, Jun. 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267166972>
- [20] Y. Mao, M. Shu, and Q. Liu, "PBN: Progressive batch normalization for DNN training on edge device," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2024, pp. 1–5. [Online]. Available: <https://api.semanticscholar.org/CorpusID:270929967>
- [21] L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu, and L. Shao, "Normalization techniques in training DNNs: Methodology, analysis and application," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 10173–10196, Aug. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:221970066>
- [22] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [23] B. Thompson et al., "Freezing subnetworks to analyze domain adaptation in neural machine translation," in *Proc. Conf. Mach. Transl.*, 2018, pp. 128–132. [Online]. Available: <https://api.semanticscholar.org/CorpusID:256461051>
- [24] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, Feb. 2019.
- [25] J. Zhang et al., "A comprehensive analysis of DAC-SDC FPGA low power object detection challenge," *Sci. China Inf. Sci.*, vol. 67, Jul. 2024, Art. no. 182401. [Online]. Available: <https://api.semanticscholar.org/CorpusID:271499268>